PROJECT REPORT

FOR

Foundations of Artificial Intelligence

MAZE AI: An Intelligent Pathfinding System

A PROJECT REPORT

Submitted by

2116240701588 – Vigneshwar M

2116240701605 – Vishnuharan R



October, 2026

BONAFIDE CERTIFICATE

Certified that this project report "**MAZE AI**: An Intelligent Pathfinding System Using BFS, DFS and A* Search with Dynamic Replanning" is the bonafide work of "**Vigneshwar M (240701588) & Vishnuharan R (240701605)**" who carried out the project work under my supervision.

SIGNATURE OF THE FACULTY INCHARGE

Submitted for the Practical Examination held on _____

SIGNATURE OF THE INTERNAL EXAMINER

TABLE OF CONTENTS

CHAPTER NO	TITLE	PAGE NO.
	ABSTRACT	
1	INTRODUCTION	5
2	LITERATURE REVIEW	7
3	PROPOSED SYSTEM	9
	3.1 ARCHITECTURE DIAGRAM	...11
4	MODULES DESCRIPTION	12
	4.1 MODULE 1	...12
	4.2 MODULE 2	...13
5	IMPLEMENTATION AND RESULTS	14
	5.1 EXPERIMENTAL SETUP	...14
	5.2 RESULTS	..16
6	CONCLUSION AND FUTURE WORK	20
7	REFERENCES	21

ABSTRACT:

MAZE AI is an interactive artificial intelligence application that demonstrates how classical search algorithms find routes through a two-dimensional grid environment. The system generates random solvable mazes, applies intelligent search to locate a path from a start cell to a goal cell, and visualizes the search process step by step so that the behaviour of each algorithm can be studied and compared.

It implements three search techniques: Breadth-First Search (BFS), Depth-First Search (DFS), and A* Search with the Manhattan distance heuristic. These algorithms support uninformed and informed search, shortest-path discovery, and heuristic-guided exploration. Each run reports three performance metrics: path length, number of nodes explored, and execution time in milliseconds.

The system follows a six-stage workflow: Generate, Normalize, Select Algorithm, Search, Validate, and Visualize/Replan. A strict wall-collision check ensures that no computed path ever passes through a wall cell, and a debug logger verifies every path before it is returned. In addition, a dynamic replanning feature allows an obstacle to be inserted while the agent is moving; the system detects the blocked route and re-runs A* from the agent's current position to the goal.

MAZE AI is developed using Python and the Flask framework for the backend, with an HTML, Tailwind CSS and JavaScript frontend that exposes the generator, solver and replanner through REST API endpoints. A self-contained command-line version (`maze_ai.py`) provides a console demonstration with colored output. Experiments on 200 random solvable mazes for each of three grid sizes showed that A* explored up to 45.7% fewer nodes than BFS while returning the same optimal path length, and that DFS returned paths up to 2.6 times longer than optimal.

The project is an academic prototype that uses randomly generated mazes rather than real-world map data. Overall, MAZE AI demonstrates how search algorithms, heuristics and replanning combine to give an agent reliable navigation in a changing environment. The system can be extended with weighted terrain, diagonal movement, additional algorithms and real map data.

1. INTRODUCTION

Navigation and route finding are fundamental problems in Artificial Intelligence. An intelligent agent placed in an environment must decide which sequence of actions will take it from its current state to a desired goal state. In robotics, video games, logistics, and automated vehicles, this decision is made by search algorithms that explore possible moves and select a suitable route while avoiding obstacles.

A maze is one of the simplest and clearest environments in which to study this problem. It can be represented as a two-dimensional grid in which each cell is either a free path or a wall. The agent moves up, down, left or right, and the task is to find a valid route from the start cell to the goal cell. Although the representation is simple, it captures the core ideas of state-space search: states, actions, goal tests, path cost, and the order in which states are expanded.

Different algorithms explore the same maze in very different ways. Breadth-First Search expands cells level by level and guarantees the shortest path in an unweighted grid. Depth-First Search follows one branch as deep as possible before backtracking and may return a long route. A* Search combines the cost already travelled with a heuristic estimate of the remaining distance, so it can reach the goal while exploring far fewer cells. Understanding these differences is difficult from textbook descriptions alone, which motivates an interactive visual tool.

MAZE AI is proposed to address this need. It is a web application and a command-line program that generates random solvable mazes, runs BFS, DFS and A*, animates the search, and reports quantitative metrics. A strict collision-checking layer guarantees that no returned path crosses a wall, and a dynamic replanning feature shows how an agent can recover when a new obstacle blocks its planned route.

The main focus of MAZE AI is to demonstrate how classical AI search algorithms behave and how they compare. The project is developed as an academic prototype using randomly generated grid environments, providing a practical setting for exploring **intelligent pathfinding and replanning** without relying on external data or hardware.

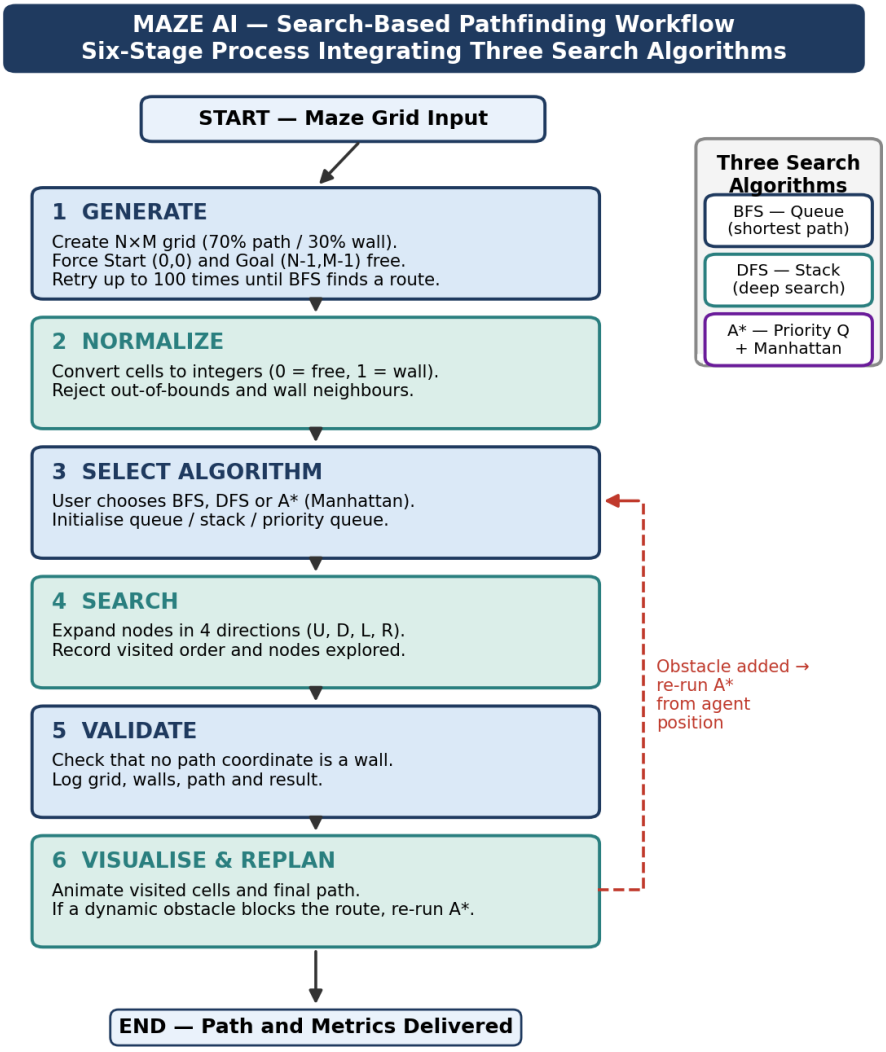


Fig 1.1 MAZE AI Search Workflow

2. LITERATURE REVIEW

Artificial Intelligence plays an important role in modern navigation and planning systems. Search algorithms provide systematic methods for exploring a space of possible states and selecting a sequence of actions that reaches a goal. MAZE AI combines several classical techniques to demonstrate how pathfinding works and how different strategies compare in terms of optimality, memory use and search effort.

Breadth-First Search (BFS):

Breadth-First Search is an uninformed search technique that explores all nodes at the current depth before moving to nodes at the next depth. It uses a First-In-First-Out (FIFO) queue. In an unweighted grid, where every move has the same cost, BFS is both complete and optimal, so the first time it reaches the goal it has found a shortest path. The idea of using breadth-first expansion to find the shortest route through a maze was described by Moore [2]. In MAZE AI, BFS is implemented with `collections.deque` and serves as the optimal baseline for comparison.

Depth-First Search (DFS):

Depth-First Search is an uninformed technique that follows one branch as far as possible before backtracking. It uses a Last-In-First-Out (LIFO) stack and requires little memory, but it does not guarantee the shortest path. Tarjan [3] showed its usefulness in graph algorithms. In MAZE AI, DFS demonstrates deep exploration and shows how a valid but non-optimal path can be produced.

A* Search Algorithm:

A* Search is an informed search algorithm introduced by Hart, Nilsson and Raphael [1]. It selects the node with the lowest value of f(n) = g(n) + h(n), where g(n) is the actual cost from the start to node n and h(n) is a heuristic estimate of the cost from n to the goal. When the heuristic is admissible, meaning it never overestimates the true cost, A* is guaranteed to return an optimal path [4]. MAZE AI implements A* with a priority queue (`heapq`) and breaks ties in favor of the node with the smaller heuristic value.

Manhattan Distance Heuristic:

The Manhattan distance between two cells is $|r1 - r2| + |c1 - c2|$. For a grid in which movement is restricted to four directions at unit cost, this value never exceeds the true remaining distance, so it is admissible and consistent. This is why A* with the Manhattan heuristic returns the same path length as BFS while exploring fewer cells.

Random Maze Generation and Solvability:

Random grids can be generated by independently marking each cell as a wall with a fixed probability. Such grids are not always solvable, so a verification step is required. MAZE AI generates a grid with a 30% wall probability, forces the start and goal cells to be free, and uses BFS to confirm that a route exists, regenerating the grid up to 100 times if necessary.

Dynamic Replanning:

In real environments, obstacles may appear after a plan has been computed. Replanning algorithms respond by computing a new route from the agent's current position. Incremental methods such as D* Lite reuse earlier search effort [5]; MAZE AI uses the simpler approach of re-running A* from the current position, which is easy to understand and sufficient for small grids.

3. PROPOSED SYSTEM

The proposed MAZE AI system is designed to make search algorithms easy to observe, compare and understand. Instead of presenting algorithms only as pseudocode, the system lets users create a maze, choose an algorithm, watch the search expand across the grid, and read quantitative results. It also allows obstacles to be placed while an agent is moving, to demonstrate replanning.

The system organizes its tasks into separate modules. The maze generation module creates random solvable grids. The search module runs BFS, DFS or A* on the grid. The validation module checks every returned path against the wall cells. The visualization module animates the explored cells and the final route, and the replanning module recomputes the route when the environment changes. These modules are available through a single web interface.

To support these functions, MAZE AI incorporates BFS, DFS and A* Search with the Manhattan heuristic. Each algorithm reports the same three metrics: path length, nodes explored, and execution time, which allows a fair comparison on identical mazes. A companion command-line program, `maze_ai.py`, provides the same algorithms with colored terminal output.

The platform uses Python with the Flask framework as the backend and an HTML, Tailwind CSS and Vanilla JavaScript frontend. The frontend communicates with the backend through JSON REST endpoints. The prototype uses randomly generated mazes only and does not depend on external data or services.

Major Components of MAZE AI:

1. **Random Maze Generation Engine** – creates an N × M grid with a 70% path and 30% wall distribution, keeps the start (0,0) and goal (N−1, M−1) cells free, and guarantees solvability through a BFS check with up to 100 attempts.
2. **Search Algorithm Engine** – implements BFS (queue), DFS (stack) and A* (priority queue with Manhattan heuristic) over a shared `Node` structure and neighbor function.
3. **Strict Collision and Path Validation** – only free in-bounds cells are returned as neighbors, and `validate_and_log_path()` rejects any path that contains a wall coordinate.
4. **Interactive Visualization Dashboard** – an interactive grid that animates visited cells in search order and then draws the final path, with controls for algorithm selection and maze size.

5. **Performance Metrics Panel** – displays path length, nodes explored and execution time (ms) for every run.
6. **Dynamic Obstacle Replanning** – detects that a new obstacle blocks the route, shows a "Path Blocked – Recalculating using A*" alert, and computes a new path from the agent's current position.
7. **Command-Line Companion (`maze_ai.py`)** – a self-contained console version with maze generation, side-by-side algorithm comparison, and a step-by-step replanning simulation.

3.1 ARCHITECTURE DIAGRAM

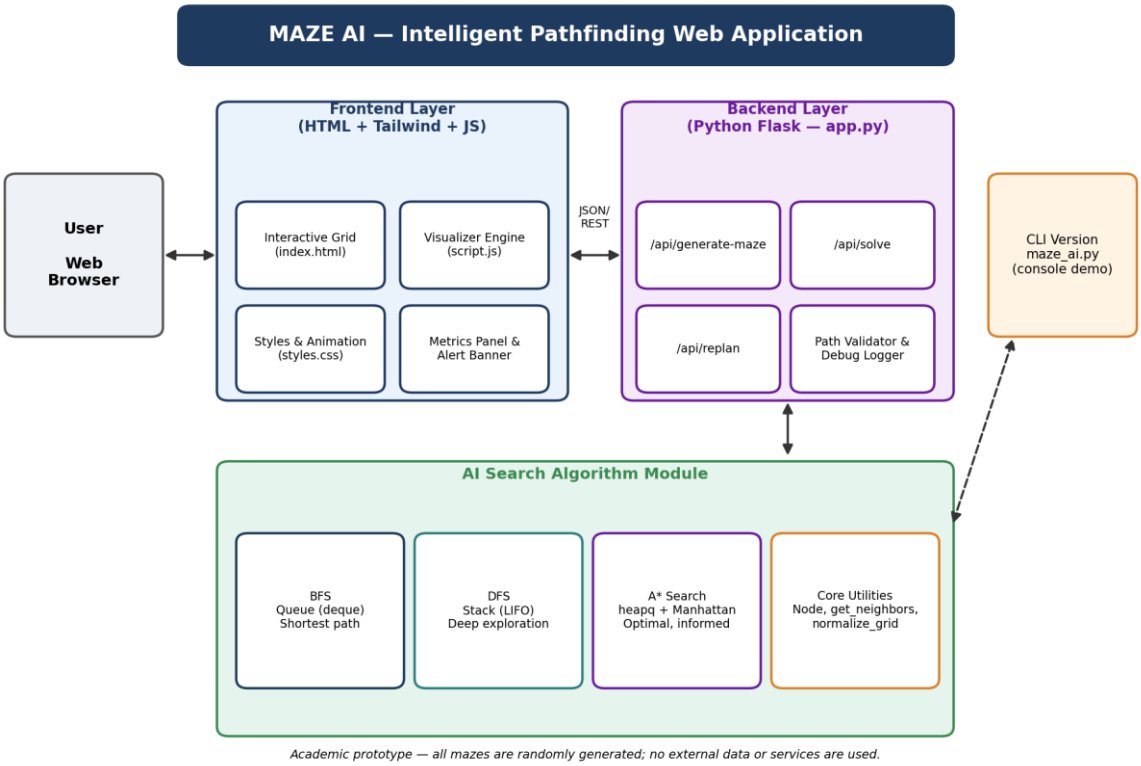


Fig 3.1 System Architecture for MAZE AI

4. MODULES DESCRIPTION

MAZE AI integrates maze generation, pathfinding, validation, visualization and replanning into a single platform. It combines three search algorithms with a strict wall-collision layer to compute routes, verify their correctness, and present the results. The application uses Python and Flask on the server, with an HTML, Tailwind CSS and JavaScript interface, and exposes its functions through the endpoints `/api/generate-maze`, `/api/solve` and `/api/replan`. The system operates as an academic prototype using randomly generated mazes.

4.1 MODULE 1: RANDOM MAZE GENERATION AND VALIDATION

The Maze Generation Module creates the environment in which every algorithm runs. A maze is stored as a two-dimensional list of integers, where 0 represents a free path and 1 represents a wall. When the user requests a new maze, the frontend sends the number of rows and columns (default 10 × 10) and an optional wall probability (default 0.30) to the `/api/generate-maze` endpoint.

For every cell except the start and goal, the generator draws a random number and marks the cell as a wall if the value is below the wall probability. The start cell (0,0) and the goal cell (rows−1, cols−1) are always set to 0. Because a random grid may be unsolvable, the module runs a BFS from start to goal after each attempt and returns the grid only if a path exists. Up to 100 attempts are made; if all fail, a fully open grid is returned as a fallback.

Before any search, the `/api/solve` endpoint calls `normalize_grid()`, which converts all cell values to the integers 0 and 1, and then forces the start and goal cells to be free. The neighbor function `get_neighbors()` returns only cells that are inside the grid and equal to 0. After a search, `validate_and_log_path()` prints the grid dimensions, wall count, wall coordinates and path coordinates, and checks that no path coordinate is a wall. If a wall is found, the path is discarded.

Main Functions:

- Generates random N × M mazes with a 70% path and 30% wall distribution.
- Keeps the start and goal cells free and guarantees a solvable maze.
- Normalizes grid values and performs strict bounds and wall checking.
- Validates every computed path and logs debug information.
- Provides the grid to the frontend as JSON for display and editing.

4.2 MODULE 2: PATHFINDING ALGORITHMS AND DYNAMIC REPLANNING

The Pathfinding Module is the core of MAZE AI. It runs the algorithm selected by the user and returns the final path, the order in which cells were visited, the number of nodes explored, the path length, and the execution time. All three algorithms use the same Node class, which stores the grid position, a parent reference, the cost g, the heuristic h and the total f = g + h.

BFS stores nodes in a `deque` and marks a cell as visited when it is added to the queue, so each cell enters the queue only once. **DFS** uses a Python list as a stack, pushes the neighbors in reversed order so that the preferred direction is explored first, and marks a cell as visited when it is popped. **A*** uses a binary heap of `(f, counter, node)` entries, where the counter keeps the ordering stable, and a `g_score` dictionary so that a cell is re-queued only when a cheaper route to it is found. The heuristic is the Manhattan distance to the goal.

When a dynamic obstacle is inserted, the frontend calls `/api/replan` with the current grid, the agent's current position and the goal. The backend clears the agent and goal cells, runs A* from the current position, validates the new path and returns it. In the command-line version, `run_dynamic_agent()` inserts an obstacle on the planned path after a chosen step, detects that the remaining route is blocked, prints "Path blocked – recalculating using A*", and continues along the new route.

Table 4.1 Characteristics of the Implemented Algorithms

Algorithm	Data structure	Type	Shortest path?	Typical behavior
BFS	Queue (deque)	Uninformed	Yes (unit cost)	Expands level by level
DFS	Stack (list)	Uninformed	No	Dives deep, long paths
A*	Priority queue (heapq)	Informed	Yes (admissible h)	Guided toward goal

Main Functions:

- Runs BFS, DFS or A* on the same maze for fair comparison.
- Records visited order for step-by-step animation of the search.
- Reports path length, nodes explored and execution time (ms).
- Rejects any path that touches a wall cell.
- Replans with A* from the agent's current position when an obstacle blocks the route.

5. IMPLEMENTATION AND RESULTS

The MAZE AI system is implemented using Python and Flask for the backend and HTML, Tailwind CSS and Vanilla JavaScript for the frontend. It integrates three search algorithms with a collision-checking and validation layer to support maze solving, comparison and replanning. The web application exposes three REST endpoints, and a separate command-line program provides the same algorithms in a console.

The system provides an interactive grid for generating mazes, choosing an algorithm and viewing the explored cells, the final path and the metrics. To evaluate the algorithms, each one was run on 200 randomly generated solvable mazes for each of three grid sizes, using the same mazes for all three algorithms. The results of these runs are reported in Section 5.2.

5.1 EXPERIMENTAL SETUP

MAZE AI is developed using the Flask web framework with the backend logic in `app.py`. The frontend consists of `templates/index.html`, `static/css/styles.css` and `static/js/script.js`. The application setup involves installing Flask, starting the server, and opening the interface in a web browser. The standalone program `maze_ai.py` requires only the Python standard library.

The experiments used the same code as the web backend: BFS, DFS and A* from `app.py`, applied to mazes generated with a 30% wall probability, the start cell at (0,0) and the goal cell at (N−1, N−1). Unsolvable grids were discarded and regenerated, exactly as in `/api/generate-maze`. Random seeds 0 to 199 were used so that the experiment can be repeated.

Additional Setup Points:

- Grid sizes tested: 10 × 10, 15 × 15 and 20 × 20.
- Number of mazes: 200 per grid size (600 mazes, 1,800 searches in total).
- Movement: four directions (up, down, left, right) with unit cost.
- Metrics: path length (steps), nodes explored, and execution time in milliseconds.

Implementation Steps:

8. Install Python 3 and Flask (`pip install flask`).
9. Place `app.py`, `maze_ai.py`, the `templates` folder and the `static` folder in one project directory.
10. Start the server with `python app.py`.

11. Open `http://127.0.0.1:5000` in a web browser.
12. Generate a maze, select an algorithm and run the search.
13. Add an obstacle during the run to observe replanning.
14. Optionally, run `python maze_ai.py` for the console demonstration.

Note: MAZE AI is an academic prototype that uses randomly generated mazes. The execution times were measured on the development machine and depend on hardware, so the node counts and path lengths are the more reliable basis for comparison.

5.2 RESULTS

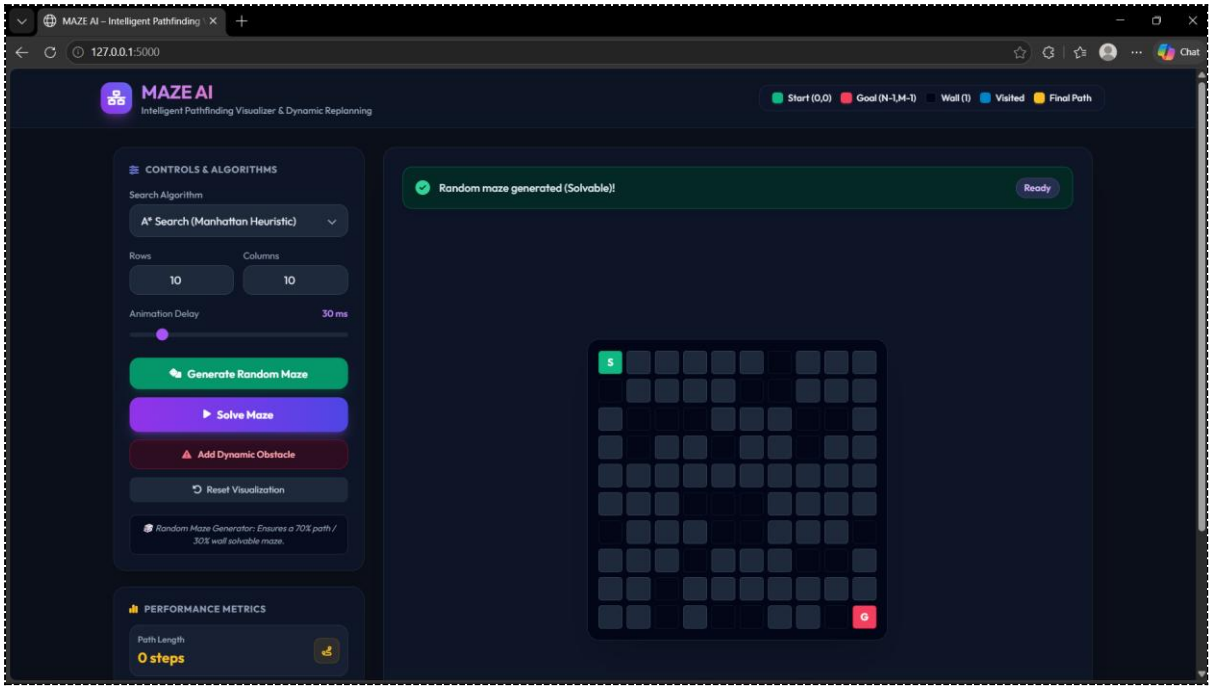


Figure 5.2.1 MAZE AI Web Interface and Random Maze Generation

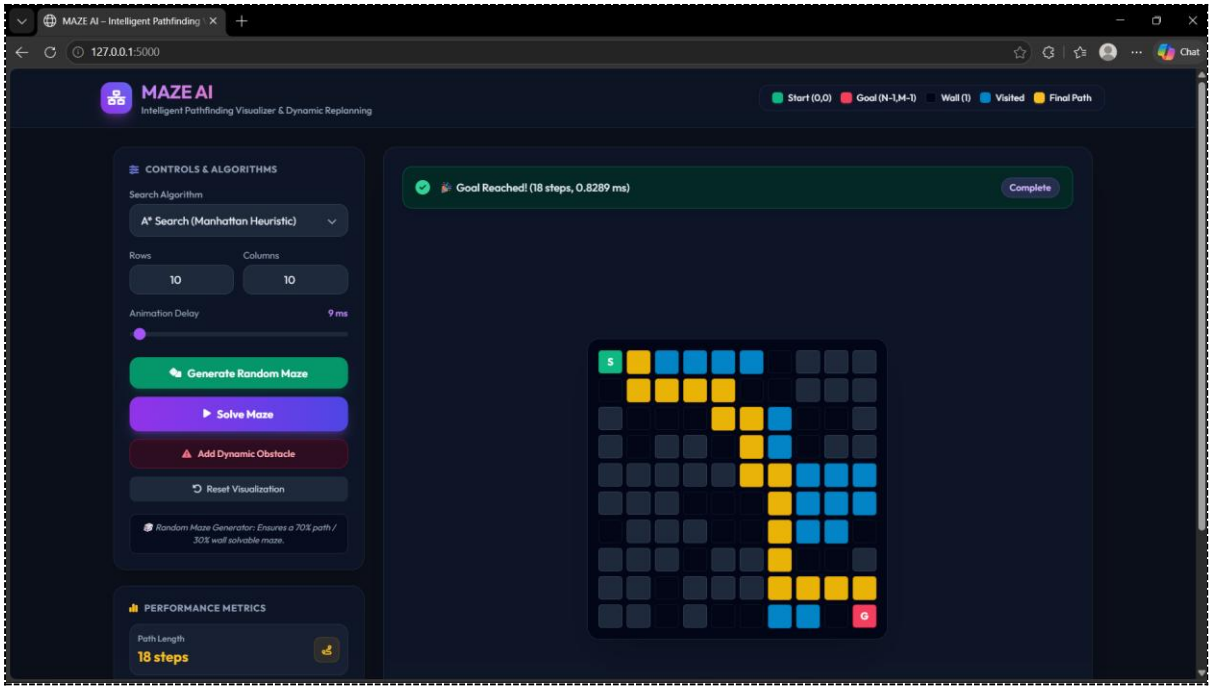


Figure 5.2.2 Algorithm Visualization and Performance Metrics Panel

Figure 5.2.3 shows BFS, DFS and A* solving the same 10 × 10 maze. Light blue cells were explored, the yellow line is the final path, S is the start and G is the goal. BFS and A* both

found a path of 18 steps, but A* explored 38 cells compared with 63 for BFS. DFS explored 60 cells but returned a path of 44 steps, more than twice the optimal length.

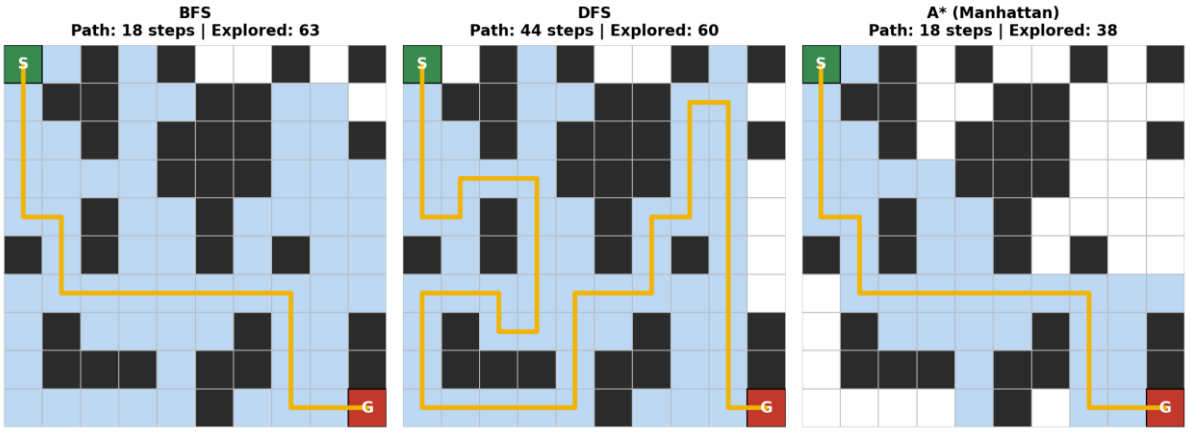


Figure 5.2.3 Search Behavior of BFS, DFS and A* on the Same Maze

Table 5.1 summarizes the averages over 200 random solvable mazes for each grid size.

Table 5.1 Average Performance over 200 Random Mazes per Grid Size

Grid size	Algorithm	Avg. path length	Avg. nodes explored	Avg. time (ms)
10 × 10	BFS	18.25	66.06	0.107
10 × 10	DFS	33.38	57.90	0.112
10 × 10	A*	18.25	48.51	0.128
15 × 15	BFS	28.56	146.58	0.219
15 × 15	DFS	63.33	123.63	0.222
15 × 15	A*	28.56	89.15	0.224
20 × 20	BFS	39.33	262.36	0.322
20 × 20	DFS	101.94	221.88	0.330
20 × 20	A*	39.33	142.50	0.295

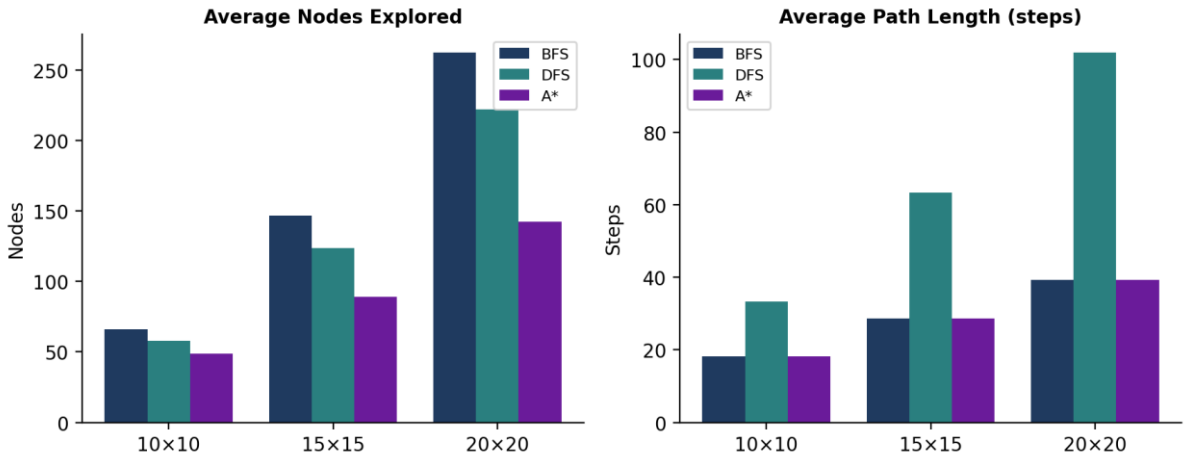


Figure 5.2.4 Average Nodes Explored and Path Length by Grid Size

Observations:

- BFS and A* returned identical average path lengths at every grid size, confirming that both are optimal in this environment.
- A* explored 26.6%, 39.2% and 45.7% fewer nodes than BFS on the 10 × 10, 15 × 15 and 20 × 20 grids respectively, so the advantage of the Manhattan heuristic grows with maze size.
- DFS explored fewer nodes than BFS on average, but its paths were about 1.8, 2.2 and 2.6 times longer than the optimal path on the three grid sizes.
- Execution times were below 0.35 ms per search for all algorithms and were too close to rank reliably; the heap operations in A* offset some of its savings in explored nodes on small grids.

Figure 5.2.5 shows the dynamic replanning feature on the same maze. The agent followed the initial A* route for two steps, and an obstacle was then placed at cell (6,3) on the remaining path. The system detected that the route was blocked, re-ran A* from the agent's position (2,0),

and found a new route of 16 steps after exploring 34 nodes, so the total journey remained 18 steps.

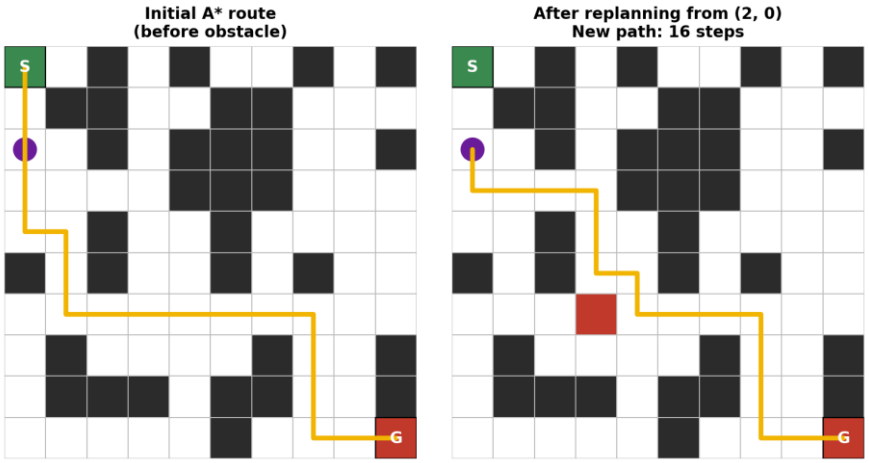


Figure 5.2.5 Dynamic Obstacle Replanning with A*

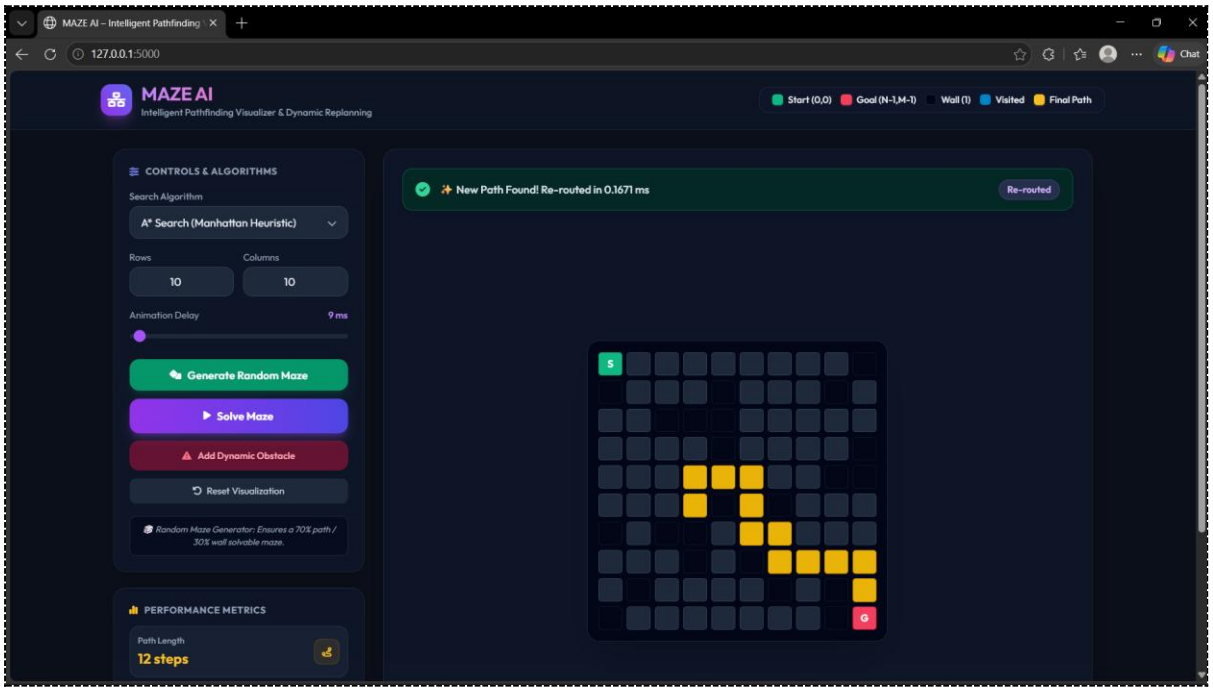


Figure 5.2.6 Path Blocked Alert and Replanning in the Web Interface

6. CONCLUSION AND FUTURE WORK

MAZE AI demonstrates the effective use of classical Artificial Intelligence search algorithms for pathfinding and replanning. It integrates BFS, DFS and A* with a strict wall-collision and validation layer, and presents the results through an interactive visual interface and a command-line program. Experiments on 600 random mazes showed that A* with the Manhattan heuristic matches the optimal path length of BFS while exploring substantially fewer nodes, that DFS is unsuitable when short paths are required, and that re-running A* from the current position allows an agent to recover when a new obstacle appears.

In the future, MAZE AI can be enhanced by adding weighted terrain, diagonal movement, and additional algorithms such as Dijkstra's algorithm, Greedy Best-First Search, Bidirectional Search and D* Lite for incremental replanning. Perfect-maze generators such as recursive backtracking can be added alongside the random generator. The system can also be extended to load real map or floor-plan data, to support multiple moving agents, and to include side-by-side animated comparison and exportable result reports.

FUTURE WORK

Further work can integrate incremental replanning methods that reuse earlier search effort, which would reduce the cost of recomputation on large grids. Moving obstacles with predicted motion, user accounts for saving mazes and results, and deployment to a public server can improve the usefulness of the application as a teaching tool for search algorithms.

7. REFERENCES

[1] P. E. Hart, N. J. Nilsson, and B. Raphael, “A formal basis for the heuristic determination of minimum cost paths,” IEEE Transactions on Systems Science and Cybernetics, vol. 4, no. 2, pp. 100–107, 1968. https://doi.org/10.1109/TSSC.1968.300136

[2] E. F. Moore, “The shortest path through a maze,” in Proceedings of an International Symposium on the Theory of Switching, Harvard University Press, pp. 285–292, 1959.

[3] R. Tarjan, “Depth-first search and linear graph algorithms,” SIAM Journal on Computing, vol. 1, no. 2, pp. 146–160, 1972.

[4] S. Russell and P. Norvig, Artificial Intelligence: A Modern Approach, 4th ed. Pearson, 2021.

[5] S. Koenig and M. Likhachev, “Fast replanning for navigation in unknown terrain,” IEEE Transactions on Robotics, vol. 21, no. 3, pp. 354–363, 2005.

[6] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, Introduction to Algorithms, 4th ed. MIT Press, 2022.

[7] Pallets Projects, “Flask Documentation.” Available: https://flask.palletsprojects.com/

[8] Python Software Foundation, “heapq — Heap queue algorithm” and “collections — Container datatypes,” Python 3 Documentation. Available: https://docs.python.org/3/